Joint Spatial Model with K-fold Gibbs Sampling

1 The big picture

Goal. Measure how well a two-response spatial model predicts data it has not seen (K-fold cross-validation), *without* refitting the model K times.

Trick. One Gibbs sampler is run. At every iteration it

(i) randomly chooses which fold $\kappa \in \{1, \ldots, K\}$ to leave out, with probabilities that depend on how well each fold is currently predicted;

(ii) updates all model parameters using only the data *outside* that fold;

(iii) re-computes how well each of the K folds is predicted by the new parameters.

These stored prediction scores are then used (in `eljpd.utils.R`) to estimate the K-fold expected log joint predictive density (ELJPD).

Model (M1). At each of $N = 3503$ map locations there are $q = 2$ responses (columns of $\mathbf{Y}$): $\log(1+\text{BA})$ modelled as Gaussian and `CNT` modelled as Poisson. Both are driven by hidden spatial surfaces that are correlated with each other (this is the "joint covariance").

Speed-up. The spatial correlation matrix is 3503×3503, so it is replaced by a *Vecchia approximation* (each location only "looks at" its $m = 20$ nearest earlier neighbours).

2 Notation

Symbol	Meaning	Code name
$N,\ q,\ p$	no. of locations, responses, covariates (3503, 2, 3)	`N, q, p`
$\mathbf{Y} \in \mathbb{R}^{N \times q}$	observed responses; y_{ij} = location i, response j	`Y`
$\mathbf{X} \in \mathbb{R}^{N \times p}$	covariates: intercept, lon, lat	`X`
$\boldsymbol{\beta} \in \mathbb{R}^{p \times q}$	regression coefficients	`beta`
$\mathbf{M} = \mathbf{X}\boldsymbol{\beta}$	prior mean of the hidden surface	`M`
$\mathbf{W} \in \mathbb{R}^{N \times q}$	hidden (latent) surfaces; row $\mathbf{w}_i$	`W`
$\boldsymbol{\Sigma} \in \mathbb{R}^{q \times q}$	covariance between the q responses	`Sigma`
$\phi,\ \nu$	Matérn range and smoothness (ν fixed at 0.5)	`phi, nu`
$\mathbf{R}_\phi \in \mathbb{R}^{N \times N}$	spatial correlation matrix	(never formed)
m	number of neighbours in the Vecchia approximation (20)	`m`
$\mathcal{N}(i)$	neighbour set of location i (earlier in ordering, incl. i)	`NNarray`
$K,\ F_k$	number of folds (10), set of locations in fold k	`K, fold`
κ	fold currently left out	`k`
θ	$(\phi, \boldsymbol{\beta}, \boldsymbol{\Sigma})$, the parameters that are saved	`theta`

3 The statistical model

3.1 Data layer: how Y depends on the hidden surface W

Given $\mathbf{W}$, all y_{ij} are independent, $y_{ij} \mid w_{ij} \sim f_j(\cdot \mid w_{ij})$. The function `loglik1` supports five choices ("par" is `family_par`, which defaults to 1 if `NA`):

Family	$\log f_j(y \mid w)$ is the log-density of	
Gaussian	$\mathcal{N}(w,\ \text{par})$ (par = variance)	**used for $j = 1$**
Poisson	$\text{Poisson}\big(e^w\big)$	**used for $j = 2$**
Binomial	$\text{Bin}\big(\text{par},\ \text{logistic}(w)\big)$	not used here
Gamma	shape = par, mean = e^w	not used here
Neg. binomial	size = par, mean = e^w	not used here

3.2 Latent layer: spatial surfaces

The hidden surfaces are a regression part plus a spatial random effect shared across responses:

$$\mathbf{W} - \mathbf{M} \ \sim\ \text{MatrixNormal}\big(\mathbf{0},\ \mathbf{R}_\phi,\ \mathbf{\Sigma}\big), \qquad \text{i.e.} \qquad \text{vec}(\mathbf{W} - \mathbf{M}) \sim \mathcal{N}\big(\mathbf{0},\ \mathbf{\Sigma} \otimes \mathbf{R}_\phi\big), \tag{1}$$

where $(\mathbf{R}_\phi)_{il} = \rho(\|s_i - s_l\|)$ is the Matérn correlation

$$\rho(d) = \frac{2^{1-\nu}}{\Gamma(\nu)}\,(\kappa_\phi d)^\nu K_\nu(\kappa_\phi d), \qquad \kappa_\phi = \frac{\sqrt{2\nu}}{\phi}, \qquad \rho(0) = 1, \tag{2}$$

and K_ν is the modified Bessel function (this is the function `matern`; it is evaluated on the log scale for numerical stability).

In plain words

$\mathbf{R}_\phi$ says *how quickly correlation dies off with distance*; $\mathbf{\Sigma}$ says *how the two responses relate at the same place* (variances and cross-correlation). Because $\nu = 0.5$ is fixed, the Matérn reduces to the simple exponential form $\rho(d) = e^{-d/\phi}$, so ϕ is a range parameter: bigger ϕ means smoother, longer-range surfaces.

3.3 Priors

$$\boldsymbol{\beta} \mid \mathbf{\Sigma} \sim \text{MatrixNormal}(\mathbf{M}_0,\ \mathbf{V}_0,\ \mathbf{\Sigma}), \quad \mathbf{\Sigma} \sim \text{Inverse-Wishart}(\delta_0,\ \mathbf{S}_0), \quad \phi \sim \text{Uniform}(0, b_\phi). \tag{3}$$

In `run_M1.R`: $\mathbf{M}_0 = \mathbf{0}$, $\mathbf{V}_0 = 100\,\mathbf{I}_p$, $\mathbf{S}_0 = \mathbf{I}_q$, $\delta_0 = q+1 = 3$ (`df_prior`), and $b_\phi = \max_{i,l} \|s_i - s_l\|/3$.

4 Vecchia approximation (C++ helper functions)

4.1 Coefficients: `vecchia_coeff_fill`

Locations are put in a max–min ordering. For location i, let $\mathcal{N}(i)$ be itself plus up to m earlier neighbours. Write $\mathbf{K}_{22}$ for the correlation matrix *among the neighbours* (excluding i) and $\mathbf{k}_i$ for the correlations between the neighbours and i. Then

$$\mathbf{b}_i = \mathbf{K}_{22}^{-1}\mathbf{k}_i, \qquad r_i = 1 - \mathbf{k}_i^\top \mathbf{K}_{22}^{-1}\mathbf{k}_i. \tag{4}$$

(Solved with a Cholesky factorisation.) The first location has no neighbours, so $r_1 = 1$.

In plain words

$\mathbf{b}_i$ are the "kriging weights" that predict location i from its neighbours, and r_i is the leftover variance (as a fraction of the total) that the neighbours cannot explain.

4.2 Approximate density of W: `vecchia_ll`

Let $\mathbf{e}_i = \mathbf{w}_i - \mathbf{m}_i$ (row i of $\mathbf{W} - \mathbf{M}$). The Vecchia approximation writes the joint density as a product of simple conditionals,

$$\mathbf{e}_i = \sum_{l \in \mathcal{N}(i) \setminus i} b_{il}\, \mathbf{e}_l + \boldsymbol{\varepsilon}_i, \qquad \boldsymbol{\varepsilon}_i \sim \mathcal{N}_q(\mathbf{0},\ r_i \boldsymbol{\Sigma}), \tag{5}$$

which gives the log-likelihood used throughout the sampler:

$$\ell(\mathbf{W} \mid \boldsymbol{\beta}, \boldsymbol{\Sigma}, \phi) = -\frac{1}{2} \sum_{i=1}^{N} \Big[q \log(2\pi r_i) + \log|\boldsymbol{\Sigma}| + \frac{1}{r_i}\, \mathbf{u}_i^\top \boldsymbol{\Sigma}^{-1} \mathbf{u}_i \Big], \qquad \mathbf{u}_i = \mathbf{e}_i - \sum_l b_{il} \mathbf{e}_l. \tag{6}$$

Code detail: with $\boldsymbol{\Sigma} = \mathbf{R}_\Sigma^\top \mathbf{R}_\Sigma$ (upper Cholesky), the quadratic form is computed by forward substitution as $\|\mathbf{R}_\Sigma^{-\top} \mathbf{u}_i\|^2$, and $\log|\boldsymbol{\Sigma}| = 2 \sum_j \log(\mathbf{R}_\Sigma)_{jj}$.

4.3 Whitened quantities: `vecchia_products`

Row i of the "whitened" matrices $\mathbf{UW}$ and $\mathbf{UX}$ is

$$(\mathbf{UW})_{i\cdot} = \frac{1}{\sqrt{r_i}} \Big(\mathbf{w}_i - \sum_l b_{il} \mathbf{w}_l \Big), \qquad (\mathbf{UX})_{i\cdot} = \frac{1}{\sqrt{r_i}} \Big(\mathbf{x}_i - \sum_l b_{il} \mathbf{x}_l \Big). \tag{7}$$

Then, approximately, $\mathbf{UW} = \mathbf{UX}\,\boldsymbol{\beta} + \mathbf{E}$ where the rows of $\mathbf{E}$ are independent $\mathcal{N}_q(\mathbf{0}, \boldsymbol{\Sigma})$.

In plain words

After this transformation the spatial dependence is removed, so $\boldsymbol{\beta}$ and $\boldsymbol{\Sigma}$ can be updated with the standard *multivariate-regression* formulas (Section 6.2).

4.4 Drawing from the spatial prior: `vecchia_prior_draw`

Given i.i.d. standard normals z_{ij}, build $\xi_{\cdot j}$ (one column per response) sequentially in the ordering:

$$\xi_{ij} = \sqrt{r_i}\, z_{ij} + \sum_{l \in \mathcal{N}(i) \setminus i} b_{il}\, \xi_{lj}. \tag{8}$$

Each column $\xi_{\cdot j}$ is a draw from $\mathcal{N}(\mathbf{0}, \mathbf{R}_\phi)$ (Vecchia version). It is used by the $\mathbf{W}$ update in Section 6.3.

5 Doing K-fold in a single run

5.1 Folds and the prediction score

Each location is randomly assigned to one of $K = 10$ folds. For a fold k, the *training* set is every location not in F_k. For a held-out site $i \in F_k$, `compute_all_fold_predictive_loglik` does the following:

1. Find the m nearest *training* sites $\mathcal{N}_k(i)$ (precomputed in `run_M1.R` via `get.knnx`). Compute Vecchia weights $\mathbf{x}_i = \mathbf{K}_{22}^{-1}\mathbf{k}_i$ and $r_i = \max(1 - \mathbf{k}_i^\top \mathbf{x}_i,\, 10^{-8})$ exactly as before.
2. Draw one fresh value of the hidden surface at the held-out site:

$$\tilde{\mathbf{w}}_i \sim \mathcal{N}_q\Big(\mathbf{m}_i + \sum_{a \in \mathcal{N}_k(i)} x_{ia}\,(\mathbf{w}_a - \mathbf{m}_a),\quad r_i\,\boldsymbol{\Sigma}\Big). \tag{9}$$

3. Score the true observation at that draw.

The fold score is

$$\ell_k(\theta) = \sum_{i \in F_k} \sum_{j=1}^{q} \log f_j\big(y_{ij} \mid \tilde{w}_{ij}\big), \qquad k = 1, \ldots, K. \tag{10}$$

In plain words

For each held-out location we ask: "using only training neighbours, what surface value would the model predict here, and how likely is the observed data under it?" Adding this up over a fold gives one number per fold, per iteration.

Note

Each ℓ_k uses a *single* random draw $\tilde{\mathbf{w}}_i$ per held-out site, drawn independently across sites and given only training neighbours. It is therefore a noisy score and not the log of the joint predictive density of the fold (Jensen: $\mathbb{E}\log f \le \log \mathbb{E} f$). Section 9 goes through the function line by line and discusses what this means for spatial correlation and for MCMC autocorrelation.

5.2 The mixture distribution being sampled ("q_{mix}")

As I read the code comments, the sampler targets a joint distribution of (θ, κ) in which:

$$\theta \mid \kappa \;\sim\; p(\theta \mid \mathbf{y}_{-F_\kappa}) \qquad \text{(posterior using only the training data)}, \tag{11}$$

$$\Pr(\kappa = k \mid \theta) \;\propto\; \exp\{-\ell_k(\theta)\} \qquad \text{(folds that are predicted } \textit{badly} \text{ are chosen } \textit{more} \text{ often)}. \tag{12}$$

The second line follows because $p(\mathbf{y}_{-F_k} \mid \theta) = p(\mathbf{y} \mid \theta) \,/\, p(\mathbf{y}_{F_k} \mid \mathbf{y}_{-F_k}, \theta)$, so $\Pr(\kappa = k \mid \theta) \propto 1/p(\mathbf{y}_{F_k} \mid \mathbf{y}_{-F_k}, \theta)$, and the code uses ℓ_k as a (noisy, single-draw) stand-in for the log of that predictive density. How good a stand-in it is, is discussed in Section 9.

Drawing κ: Gumbel-max trick. Instead of normalising $e^{-\ell_k}$ (which can overflow), draw $U_l \sim \text{Unif}(0,1)$ and take

$$\kappa = \arg\max_l \Big\{ -\ell_l(\theta) - \log\big(-\log U_l\big)\Big\}. \tag{13}$$

This gives exactly $\Pr(\kappa = k) = e^{-\ell_k} / \sum_l e^{-\ell_l}$.

6 One iteration of the sampler (`gibbs_kfold_joint_cpp`)

The state is $(\phi, \boldsymbol{\beta}, \boldsymbol{\Sigma}, \mathbf{W})$ plus κ. Each iteration, in the order written in the code:

Algorithm: one Gibbs iteration

1. **Fold:** draw $\kappa \mid \theta$ by the Gumbel-max rule; mark F_κ as held out.
2. ϕ**:** Metropolis–Hastings step given $(\mathbf{W}, \boldsymbol{\beta}, \boldsymbol{\Sigma})$ (`update_phi`)
3. $\boldsymbol{\Sigma}$ **then** $\boldsymbol{\beta}$**:** conjugate draws given $\mathbf{W}, \phi$ (`update_Sigma`, `update_beta`)
4. **W:** elliptical slice sampling, one column at a time, using the likelihood of the *training* sites only (`update_W`)
5. **Scores:** recompute $\ell_1, \ldots, \ell_K$ at the new state (`compute_all_fold_predictive_loglik`)
6. **Save** ϕ, $\boldsymbol{\beta}$, upper triangle of $\boldsymbol{\Sigma}$, all ℓ_k, and κ.

In plain words

Steps 2–4 are an ordinary Bayesian spatial-model update, except that the data in fold κ are ignored. That is exactly what makes $\theta \mid \kappa$ the "leave-fold-κ-out" posterior.

6.1 Step 2: updating ϕ (`update_phi`)

Random-walk Metropolis–Hastings on ϕ with target $\propto \ell(\mathbf{W} \mid \boldsymbol{\beta}, \boldsymbol{\Sigma}, \phi) \times \mathrm{Unif}(0, b_\phi)$.

Proposal. $\phi^* \sim \mathcal{N}(\phi, \tau)$ truncated to $(0, b_\phi)$, where $\tau =$ `tuning_phi` $= 0.01$ is a *variance*. (The code simply re-draws until the value lands in the interval.) The Vecchia coefficients are recomputed at ϕ^*.

Acceptance. Since the proposal is truncated, it is not symmetric, so a correction is needed. With $s = \sqrt{\tau}$ and

$$Z(x) = \Phi\Big(\tfrac{b_\phi - x}{s}\Big) - \Phi\Big(\tfrac{-x}{s}\Big), \tag{14}$$

accept ϕ^* with probability $\min(1, e^{\log r})$, where

$$\log r = \ell(\mathbf{W} \mid \boldsymbol{\beta}, \boldsymbol{\Sigma}, \phi^*) - \ell(\mathbf{W} \mid \boldsymbol{\beta}, \boldsymbol{\Sigma}, \phi) + \log Z(\phi) - \log Z(\phi^*). \tag{15}$$

(The uniform prior is constant inside $(0, b_\phi)$ so it cancels.) The overall acceptance rate is returned as `acc`.

6.2 Step 3: updating $\boldsymbol{\Sigma}$ and $\boldsymbol{\beta}$ (`update_Sigma`, `update_beta`)

Using the whitened matrices, this is Bayesian multivariate linear regression $\mathbf{UW} = \mathbf{UX}\,\boldsymbol{\beta} + \mathbf{E}$ with a conjugate matrix-normal–inverse-Wishart prior. Define

$$\widetilde{\mathbf{V}} = \big(\mathbf{UX}^\top \mathbf{UX} + \mathbf{V}_0^{-1}\big)^{-1}, \tag{16}$$

$$\widetilde{\mathbf{M}} = \widetilde{\mathbf{V}}\big(\mathbf{UX}^\top \mathbf{UW} + \mathbf{V}_0^{-1}\mathbf{M}_0\big), \tag{17}$$

$$\widetilde{\mathbf{S}} = \mathbf{S}_0 + \mathbf{UW}^\top \mathbf{UW} + \mathbf{M}_0^\top \mathbf{V}_0^{-1}\mathbf{M}_0 - \widetilde{\mathbf{M}}^\top \widetilde{\mathbf{V}}^{-1}\widetilde{\mathbf{M}}. \tag{18}$$

Then, in this order (so $\boldsymbol{\beta}$ is integrated out when drawing $\boldsymbol{\Sigma}$):

$$\boldsymbol{\Sigma} \mid \mathbf{W}, \phi \sim \text{Inverse-Wishart}\big(\delta_0 + N,\ \widetilde{\mathbf{S}}\big), \tag{19}$$

$$\boldsymbol{\beta} \mid \boldsymbol{\Sigma}, \mathbf{W}, \phi \sim \text{MatrixNormal}\big(\widetilde{\mathbf{M}},\ \widetilde{\mathbf{V}},\ \boldsymbol{\Sigma}\big). \tag{20}$$

Code details. The inverse-Wishart draw (`riwish_cpp`) uses the Bartlett decomposition. The $\boldsymbol{\beta}$ draw is $\boldsymbol{\beta} = \widetilde{\mathbf{M}} + \mathbf{R}_{\tilde{V}}^\top \mathbf{Z}\,\mathbf{R}_\Sigma$ with $\mathbf{Z}$ a $p \times q$ matrix of standard normals and $\mathbf{R}_{\tilde{V}}, \mathbf{R}_\Sigma$ the upper Cholesky factors of $\widetilde{\mathbf{V}}, \boldsymbol{\Sigma}$.

Binomial responses (not active in this run). After drawing $\boldsymbol{\Sigma}^*$, the code splits it into standard deviations and correlations, keeps the correlations, and replaces the standard deviation of any Binomial response by a fixed value (taken from the initial $\boldsymbol{\Sigma}$). This is the usual device to keep the logistic scale identifiable. Here both families are Gaussian/Poisson, so $\boldsymbol{\Sigma}$ is simply the inverse-Wishart draw.

In plain words

Given the hidden surfaces, learning the regression coefficients and the between-response covariance is a textbook conjugate update; the Vecchia whitening is what lets us treat the rows as independent.

6.3 Step 4: updating W (`update_W`)

$\mathbf{W}$ cannot be drawn directly because the Poisson (and Gaussian-with-fixed-variance) likelihood is not conjugate. The code uses *elliptical slice sampling* (ESS), one column j at a time.

Conditional prior for column j. Let "$-j$" mean "all columns except j". From the multivariate-normal prior,

$$\mathbf{c}_j = \boldsymbol{\Sigma}_{-j,-j}^{-1}\boldsymbol{\Sigma}_{-j,j}, \quad \boldsymbol{\mu}_j = \mathbf{M}_{\cdot j} + (\mathbf{W}_{\cdot,-j} - \mathbf{M}_{\cdot,-j})\,\mathbf{c}_j, \quad \sigma^2_{j|-j} = \Sigma_{jj} - \boldsymbol{\Sigma}_{j,-j}\boldsymbol{\Sigma}_{-j,-j}^{-1}\boldsymbol{\Sigma}_{-j,j}, \tag{21}$$

so that, a priori, $\mathbf{W}_{\cdot j} \mid \mathbf{W}_{\cdot,-j} \sim \mathcal{N}\big(\boldsymbol{\mu}_j,\ \sigma^2_{j|-j}\mathbf{R}_\phi\big)$.

ESS move. With $\mathbf{w}^{\text{cur}} = \mathbf{W}_{\cdot j}$ and a prior draw $\xi_{\cdot j}$ from Section 4.4:

1. Set a slice height $\log y = L(\mathbf{w}^{\text{cur}}) + \log u$, $u \sim \text{Unif}(0,1)$, where
$$L(\mathbf{w}) = \sum_{i \notin F_\kappa} \log f_j(y_{ij} \mid w_i) \qquad \text{(training sites only)}. \tag{22}$$
2. Pick an angle $\gamma \sim \text{Unif}(0, 2\pi)$ and bracket $[\gamma - 2\pi, \gamma]$.
3. Propose
$$\mathbf{w}^{\text{new}} = \boldsymbol{\mu}_j + (\mathbf{w}^{\text{cur}} - \boldsymbol{\mu}_j)\cos\gamma + \sigma_{j|-j}\,\xi_{\cdot j}\sin\gamma. \tag{23}$$
4. If $L(\mathbf{w}^{\text{new}}) > \log y$, accept. Otherwise shrink the bracket towards 0 (set $\gamma_{\min} = \gamma$ if $\gamma < 0$, else $\gamma_{\max} = \gamma$), draw a new γ uniformly in the bracket and repeat (at most 1000 tries).

In plain words

ESS proposes a new surface that is a "rotation" between the current surface and a fresh draw from the prior. It needs no tuning, and the move is always accepted eventually because the bracket shrinks towards the current value ($\gamma = 0$). Because L only counts the training sites, the hidden surface at held-out sites is shaped by the prior and its neighbours, not by their own observations; this is how fold κ is genuinely "left out".

6.4 What is returned

For S iterations the function returns:

- `samples` (= `theta`): an $S \times (1 + pq + q(q+1)/2)$ matrix. Here $1 + 6 + 3 = 10$ columns: ϕ; then $\boldsymbol{\beta}$ column by column ($\beta_{\cdot 1}$, $\beta_{\cdot 2}$); then the upper triangle of $\boldsymbol{\Sigma}$ ($\Sigma_{11}, \Sigma_{12}, \Sigma_{22}$).
- `fold_loglik`: $S \times K$ matrix of the scores ℓ_k.
- `folds`: the sequence of left-out folds $\kappa^{(s)}$.

- `acc`: acceptance rate of the ϕ update.
- `state`: final $\mathbf{W}, \boldsymbol{\beta}, \boldsymbol{\Sigma}, \phi$.

7 `run_M1.R`: the driver script

7.1 Settings

Quantity	Value	R name
Folds K	10 (random assignment)	`k.folds, fold_id`
Neighbours m	20	`m`
Time slice used	$t = 140$ (one month of data, $N = 3503$ sites)	`t.index, N.s`
Covariates $\mathbf{X}$	$(1,\ \text{lon},\ \text{lat})$	`X`
Responses $\mathbf{Y}$	$\big(\log(1+\text{BA}),\ \text{CNT}\big)$	`Y`
Families	Gaussian, Poisson	`family`
Matérn smoothness	$\nu = 0.5$, fixed	`nu`
Range prior upper bound	$b_\phi = \max \text{dist}/3$	`b_phi`
Proposal variance for ϕ	0.01	`tuning.phi`
Gibbs iterations	$S_{\max} = 1000$; checkpoints $S = 100, 200, \ldots, 1000$	`S_max, S_vals`

7.2 What the script does, step by step

1. **Load code and data.** Sources the Vecchia helpers, compiles the C++ file with `sourceCpp`, loads `data_full.RData`, and keeps the $N = 3503$ rows for time $t = 140$.
2. **Order and neighbours.** Reorders locations with a max–min ordering (`max_min`), then builds the $N \times (m+1)$ neighbour matrix `NNarray` (each row: the location and its m nearest earlier neighbours).
3. **Test neighbours for each fold.** For each fold k, finds the m nearest training sites of every held-out site (`get.knnx`); this is what the prediction score uses.
4. **Priors.** $\mathbf{M}_0 = \mathbf{0}$, $\mathbf{V}_0 = 100\mathbf{I}_3$, $\mathbf{S}_0 = \mathbf{I}_2$, $\delta_0 = 3$.
5. **Starting values.**

$$\mathbf{W}^{(0)} = \big(y_{\cdot 1},\ \log(1 + y_{\cdot 2})\big), \qquad \boldsymbol{\Sigma}^{(0)} = \widehat{\text{Cov}}(\mathbf{W}^{(0)}), \qquad \boldsymbol{\beta}^{(0)} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{W}^{(0)}, \tag{24}$$

   and $\phi^{(0)} = 2$, $\nu = 0.5$. (The variogram-based initialisation of ϕ, ν is commented out.)
6. **Run the sampler.** Calls `gibbs_kfold_joint_cpp` for 1000 iterations and reports the acceptance rate and how many distinct folds were visited.
7. **ELJPD at checkpoints.** For each $S \in \{100, \ldots, 1000\}$, uses the first S rows of `fold_loglik` and `theta` (only the columns that actually vary) to compute `eljpd_hat`, its asymptotic standard error, the effective sample size of the ELPD estimate and the minimum ESS of the parameters (functions `tuple_function`, `eljpd.details`).
8. **Save** the history table, acceptance rate and settings to `M1_outputs.RData`.

What the final estimate is. The quantity of interest is the K-fold ELJPD,

$$\text{ELJPD} = \sum_{k=1}^{K} \log p\big(\mathbf{y}_{F_k} \mid \mathbf{y}_{-F_k}\big), \tag{25}$$

the log predictive density of each held-out fold given the rest, summed over folds. The importance-sampling-type estimator that turns the saved scores ℓ_k and the chain into this number is inside

`eljpd.utils.R`, which was not among the uploaded files, so I have described it only through how it is called.

8 `update_parameters_joint.R`: the R versions of the updates

This file contains plain-R versions of the same four conditional updates. The mathematics is identical to Sections 6.1–6.3; the table lists what is different.

R function	C++ equivalent	Difference
`update.W`	`update_W`	Same ESS algorithm. The R version scores *all* observed sites (no fold mask) through `log.likelihood()` and recomputes $\mathbf{\Sigma}^{-1}_{-j,-j}$ inside the loop. The C++ version only counts training sites ($i \notin F_\kappa$).
`update.phi`	`update_phi`	Same MH step. R writes the uniform prior explicitly (`dunif`) and uses `rtruncnorm` / `dtruncnorm`; C++ uses re-draw-until-inside and the normalising constants $Z(\cdot)$ directly (the result is the same acceptance ratio).
`update.Sigma`	`update_Sigma`	Same: $\widetilde{\mathbf{S}}$, inverse-Wishart with $\delta_0 + N$ degrees of freedom, and fixing the standard deviation for Binomial responses. R also returns the Cholesky factor of $\mathbf{\Sigma}^{-1}$.
`update.beta`	`update_beta`	Same matrix-normal draw $\boldsymbol{\beta} = \widetilde{\mathbf{M}} + \mathbf{R}_{\tilde{V}}^{\top}\mathbf{Z}\mathbf{R}_\Sigma$.

In plain words

The R file is the standard (non-K-fold) sampler's building blocks; the C++ file contains the same blocks plus the three new K-fold ingredients: the fold draw κ, the training mask inside the $\mathbf{W}$ update, and the fold scores ℓ_k.

9 Line by line: `compute_all_fold_predictive_loglik`

9.1 What the function is for

This function produces the K numbers $\ell_1, \ldots, \ell_K$ of Section 5.1. It is called once before the loop (at the starting values) and once at the end of every Gibbs iteration. The numbers are stored in `fold_ll`, which (a) is used at the start of the *next* iteration to draw κ, and (b) is saved in `fold_loglik` for the ELJPD estimate.

Inputs.

Argument	Meaning
`Y`	observed responses, $N \times q$ (in the max–min ordering)
`X`, `beta`	covariates and coefficients, so the prior means are $\mathbf{M} = \mathbf{X}\boldsymbol{\beta}$
`W`	current hidden surfaces at *all* N sites, $N \times q$ (the latest Gibbs draw)
`cholSigma`	upper Cholesky factor $\mathbf{R}_\Sigma$ of $\mathbf{\Sigma}$, i.e. $\mathbf{\Sigma} = \mathbf{R}_\Sigma^{\top}\mathbf{R}_\Sigma$
`locs`	coordinates (longitude, latitude), $N \times 2$
`test_nn_list`	list of length K. Entry k holds `test_idx` (the held-out sites F_k) and `nn_idx` (for each held-out site, its m nearest *training* sites; all indices 0-based)
`phi`, `nu`	Matérn parameters at the current iteration
`fam`, `par`	response families and their extra parameters (Section 3.1)
`out`	output vector of length K (filled with $\ell_1, \ldots, \ell_K$)

9.2 The code, block by block

Block A: set-up.

```
const int K = test_nn_list.size(), q = Y.n_cols, d = locs.ncol();
const arma::mat M = X * beta;
out.assign(K, 0.0);
```

- `K` is the number of folds (length of the list), `q`= 2 responses, `d`= 2 coordinates.
- `M`= $\mathbf{X}\boldsymbol{\beta}$ is the $N \times q$ matrix whose row i is $\mathbf{m}_i$, the regression part of the hidden surface at site i. It is computed once and reused for all sites.
- `out` is reset to K zeros.

Block B: loop over folds and unpack the list.

```
for (int k = 0; k < K; ++k) {
  List f_info = test_nn_list[k];
  IntegerVector t_idx = f_info["test_idx"];
  IntegerMatrix nn_idx = f_info["nn_idx"];
  const int n_test = t_idx.size(), m = nn_idx.ncol();
```

- `t_idx` lists the held-out sites $t_1 < t_2 < \cdots < t_n$ of fold F_k (n = `n_test`, about $N/K \approx 350$).
- Row i of `nn_idx` lists $\mathcal{N}_k(t_i)$: the $m = 20$ training sites (sites *not* in F_k) closest to t_i. These were precomputed in `run_M1.R`.

Block C: work space.

```
arma::mat K22(m, m);
arma::vec K21(m);
double ll_fold = 0.0;
```

`K22` and `K21` will hold the spatial correlations needed for one held-out site and are reused. `ll_fold` will accumulate ℓ_k.

Block D: loop over held-out sites.

```
for (int i = 0; i < n_test; ++i) {
  const int site_i = t_idx[i];
```

From here until Block I everything is done *separately for each held-out site* t_i (written `site_i`). Nothing computed for one held-out site is passed to the next.

Block E: correlations between the site and its neighbours.

```
for (int a = 0; a < m; ++a) {
  int n_a = nn_idx(i, a);
  for (int b = 0; b <= a; ++b) {
    int n_b = nn_idx(i, b);
    double ss = 0.0;
    for (int c = 0; c < d; ++c) {
      double df = locs(n_a, c) - locs(n_b, c);
      ss += df * df;
    }
    double val = matern(std::sqrt(ss), phi, nu);
    K22(a, b) = val; K22(b, a) = val;
  }
  double ss = 0.0;
  for (int c = 0; c < d; ++c) {
    double df = locs(n_a, c) - locs(site_i, c);
    ss += df * df;
  }
  K21(a) = matern(std::sqrt(ss), phi, nu);
}
```

The innermost `c`-loop is just the Euclidean distance $\sqrt{\sum_c (\text{coordinate difference})^2}$. The block fills

$$(\mathbf{K}_{22})_{ab} = \rho\big(\|s_{n_a} - s_{n_b}\|\big), \qquad (\mathbf{K}_{21})_a = \rho\big(\|s_{n_a} - s_{t_i}\|\big), \tag{26}$$

with ρ the Matérn correlation at the *current* ϕ. Only the lower triangle of `K22` is computed (`b <= a`); the other half is copied. `K22` is the correlation *among the m training neighbours*; `K21` is the correlation between the neighbours and the held-out site.

Block F: kriging weights and leftover variance.

```
arma::mat R = arma::chol(K22, "upper");
arma::vec y = arma::solve(arma::trimatl(R.t()), K21);
arma::vec x = arma::solve(arma::trimatu(R), y);
double dot = arma::dot(x, K21);
double r_i = 1.0 - dot;
if (r_i < 1e-8) r_i = 1e-8;
```

- `R` is the upper Cholesky factor, $\mathbf{K}_{22} = \mathbf{R}^\top \mathbf{R}$.
- `y` solves $\mathbf{R}^\top \mathbf{y} = \mathbf{K}_{21}$ (forward substitution) and `x` solves $\mathbf{R}\mathbf{x} = \mathbf{y}$ (back substitution). Together, $\mathbf{x} = \mathbf{K}_{22}^{-1}\mathbf{K}_{21}$. (This `y` is a temporary vector, not the data.)
- `dot`$= \mathbf{K}_{21}^\top \mathbf{K}_{22}^{-1}\mathbf{K}_{21}$ and $r_i = 1 - \text{dot}$.
- The last line sets a floor $r_i \geq 10^{-8}$ to avoid a zero or negative variance from rounding.

These are the same Vecchia quantities $\mathbf{b}_i$ and r_i as in Section 4.1, but built from *training* neighbours only.

Block G: conditional mean of the hidden surface at the held-out site.

```
arma::rowvec mean_i = M.row(site_i);
for (int a = 0; a < m; ++a) {
  int n_a = nn_idx(i, a);
  mean_i += x(a) * (W.row(n_a) - M.row(n_a));
}
```

$$\boldsymbol{\mu}_i^{\text{pred}} = \mathbf{m}_i + \sum_{a=1}^{m} x_a \left(\mathbf{w}_{n_a} - \mathbf{m}_{n_a}\right) \qquad (1 \times q). \tag{27}$$

Start from the regression mean at the held-out site, then add a weighted average of how far the *current Gibbs draw of* $\mathbf{W}$ at each neighbour sits above or below *its* regression mean. The weights x_a are the same for both responses: because the model is $\boldsymbol{\Sigma} \otimes \mathbf{R}_\phi$ (separable), $\boldsymbol{\Sigma}$ cancels out of the conditional mean.

Block H: one random draw of the hidden surface.

```
arma::vec z(q);
for (int j = 0; j < q; ++j) z(j) = norm_rand();
arma::rowvec w_draw = mean_i + std::sqrt(r_i) * (z.t() * cholSigma);
```

With $\mathbf{z} \sim \mathcal{N}_q(\mathbf{0}, \mathbf{I}_q)$,

$$\tilde{\mathbf{w}}_i = \boldsymbol{\mu}_i^{\text{pred}} + \sqrt{r_i}\, \mathbf{z}^\top \mathbf{R}_\Sigma \implies \tilde{\mathbf{w}}_i \sim \mathcal{N}_q\big(\boldsymbol{\mu}_i^{\text{pred}},\ r_i \boldsymbol{\Sigma}\big). \tag{28}$$

Multiplying standard normals by $\mathbf{R}_\Sigma$ gives covariance $\mathbf{R}_\Sigma^\top \mathbf{R}_\Sigma = \boldsymbol{\Sigma}$, which correlates the two responses at this site; r_i scales the spread by how much the neighbours fail to explain. **Only one draw is made.**

Block I: score the observed data at that draw.

```
for (int j = 0; j < q; ++j) {
  ll_fold += loglik1(fam[j], par[j], Y(site_i, j), w_draw(j));
}
}   // end of loop over held-out sites
out[k] = ll_fold;
}   // end of loop over folds
```

For each response j the code adds $\log f_j(y_{t_i j} \mid \tilde{w}_{ij})$ (Gaussian or Poisson here) to the running total. After the last site of fold k, the total is stored as `out[k]`$= \ell_k$.

9.3 Everything in one formula

$$\ell_k = \sum_{i \in F_k} \sum_{j=1}^{q} \log f_j\big(y_{ij} \mid \tilde{w}_{ij}\big), \qquad \tilde{\mathbf{w}}_i \overset{\text{independent over } i}{\sim} \mathcal{N}_q\Big(\mathbf{m}_i + \sum_{a \in \mathcal{N}_k(i)} x_{ia}(\mathbf{w}_a - \mathbf{m}_a),\ r_i \boldsymbol{\Sigma}\Big) \tag{29}$$

9.4 Does it account for spatial correlation?

Kind of dependence	Included?	How
Held-out site ↔ its training neighbours	Yes	Weights x_{ia} and variance r_i come from the Matérn correlation at the current ϕ; the neighbours' current $\mathbf{W}$ values enter the mean.
Response 1 ↔ response 2 at the same site	Yes	Through $\boldsymbol{\Sigma}$ in the draw $\mathbf{z}^\top \mathbf{R}_\Sigma$.
Only $m = 20$ neighbours	Approx.	Vecchia truncation (same as in the model).
Held-out site ↔ *other held-out sites*	**No**	Neighbours are training sites only, and each $\tilde{\mathbf{w}}_i$ is drawn independently of the others.
Surface already sampled at held-out sites in `W`	**No**	Rows of `W` at held-out sites are never read here; a fresh draw replaces them.

Consequence. The score is built from per-site conditionals given the training data. Two held-out sites that sit next to each other should have very similar hidden values, but here they receive unrelated draws. So ℓ_k is *not* the log of the joint predictive density $p(\mathbf{y}_{F_k} \mid \mathbf{y}_{-F_k}, \theta)$ of the whole fold; it is a sum of single-site, single-draw terms.

A second (separate) issue, from my reading. In an iteration where fold κ is held out, `W` has been updated with the data of all other folds $l \neq \kappa$. When the function scores fold $l \neq \kappa$, the neighbours' `W` values were therefore partly shaped by $\mathbf{y}_{F_l}$ itself (hidden surfaces are spatially correlated, so nearby values carry that information). Those scores are then optimistic compared with ℓ_κ, which is computed from a state that never saw $\mathbf{y}_{F_\kappa}$. Because κ is drawn with probability $\propto e^{-\ell}$, this pushes the chain to stay on the current fold. Section 10.5 discusses this and an exact alternative; please check what the method intends.

9.5 Does it account for MCMC autocorrelation?

Not in this function, and it is not meant to. The function has no memory: it is a function of the current state $(\phi, \boldsymbol{\beta}, \boldsymbol{\Sigma}, \mathbf{W})$ plus fresh random numbers. MCMC autocorrelation lives in the *sequence* of states $(\theta^{(s)}, \mathbf{W}^{(s)}, \kappa^{(s)})$ generated by the Gibbs sampler from q_{mix}; it should be dealt with at the *estimation* stage (long-run variance, batch means, ESS), not inside ℓ_k. In `run_M1.R` this is what `eljpd.details` reports (`asymp_se`, `ess_elpd`, `minESS_xi`) and the package `mcmcse` is loaded; I cannot confirm how it does so because `eljpd.utils.R` was not uploaded.

What this function does to the autocorrelation. Write the stored score as

$$\ell_k^{(s)} = g_k^{(s)} + \varepsilon_k^{(s)}, \qquad g_k^{(s)} = \mathbb{E}\big[\ell_k^{(s)} \mid \text{state}^{(s)}\big], \tag{30}$$

where $g_k^{(s)}$ is the part that depends on the chain's state (and so is autocorrelated) and $\varepsilon_k^{(s)}$ is the noise from the fresh draws $\tilde{\mathbf{w}}_i$, independent from one iteration to the next. For $h \geq 1$,

$$\operatorname{Corr}\big(\ell_k^{(s)}, \ell_k^{(s+h)}\big) \ \approx\ \operatorname{Corr}\big(g_k^{(s)}, g_k^{(s+h)}\big) \times \frac{\operatorname{Var}(g_k)}{\operatorname{Var}(g_k) + \operatorname{Var}(\varepsilon_k)}. \tag{31}$$

If the noise variance is large compared with how much g_k moves between iterations, the stored scores look uncorrelated and the estimated ESS is close to S, *even though the chain itself is autocorrelated*. This is the most likely reason for the ESS $\approx S$ you saw. A large ESS here is therefore not evidence of good mixing. (The approximation ignores the small feedback of ε_k into the next draw of κ.)

Noise also feeds back into the chain. Next iteration, κ is drawn with probabilities $\propto e^{-\ell_k}$ using these noisy ℓ_k. So the noise changes which training set is used, which is a second route by which it affects the chain, and the draw of $\kappa \mid \theta$ is not exactly the intended conditional (the exponential of a noisy log-score is not an unbiased version of $1/p(\mathbf{y}_{F_k} \mid \mathbf{y}_{-F_k}, \theta)$).

Quick checks. (i) Compare `acf` of the columns of `theta` with `acf` of the columns of `fold_loglik`; clear autocorrelation in the first but none in the second confirms the diagnosis. (ii) Look at `table(folds)` to see how often each fold is picked. (iii) Repeat the run with the score averaged over several draws and see whether the ESS of `fold_loglik` moves toward that of `theta`.

9.6 What a version that respects both would need

These are suggestions to discuss, not something the current code does.

1. **Spatial correlation between held-out sites.** Within a fold, process held-out sites in order $t_1 < \cdots < t_n$; let the neighbours of t_i be the m nearest sites among the training sites *and the earlier held-out sites*, and store each new draw $\tilde{\mathbf{w}}_{t_i}$ so later sites can use it. This is a joint Vecchia draw of $\mathbf{W}_{F_k} \mid \mathbf{W}_{-F_k}$.

2. **A proper (not single-draw) score.** Draw R joint replicates $\tilde{\mathbf{W}}_{F_k}^{(r)}$ and use a log-mean-exp, e.g. for a joint fold density

$$\ell_k \approx \log\Big\{\frac{1}{R}\sum_{r=1}^{R}\exp\Big(\sum_{i\in F_k}\sum_{j}\log f_j\big(y_{ij} \mid \tilde{w}_{ij}^{(r)}\big)\Big)\Big\}. \tag{32}$$

   With about 350 sites per fold the inner sum varies enormously across r, so this naive average is dominated by a few replicates; a large R or a smarter proposal may be needed. If the target is instead a per-site (pointwise) predictive density, the log-mean-exp is applied site by site.

3. **A clean out-of-sample state.** Make sure the surface used to score fold l has not been conditioned on $\mathbf{y}_{F_l}$.

Which of these are needed depends on exactly how the joint predictive and q_{mix} are defined in your paper.

10 The main function `gibbs_kfold_joint_cpp` and the full conditionals of q_{mix}

10.1 What the function does overall

It takes the data, the neighbour structure, the folds, the priors and starting values; checks and converts them; and then runs $S =$ `niters` iterations of a Gibbs sampler on the mixture distribution q_{mix}. It returns the stored draws of θ, the K fold scores for every iteration, the sequence of left-out folds and the acceptance rate of ϕ.

10.2 The target and the full conditionals it requires

Keep the hidden surface $\mathbf{W}$ (at all N sites) in the state. The distribution the code is built to sample is

$$q_{\text{mix}}(\theta, \mathbf{W}, \kappa = k) \;\propto\; p(\theta)\, p_V(\mathbf{W} \mid \theta) \prod_{i \notin F_k} f(y_i \mid \mathbf{w}_i), \tag{33}$$

where p_V is the Vecchia matrix-normal prior of Section 3 and $f(y_i \mid \mathbf{w}_i) = \prod_j f_j(y_{ij} \mid w_{ij})$. Two facts follow directly:

- Given $\kappa = k$, $(\theta, \mathbf{W}) \sim p(\theta, \mathbf{W} \mid \mathbf{y}_{-F_k})$: the posterior that uses only the training data.
- The marginal of κ is $\propto p(\mathbf{y}_{-F_k}) = p(\mathbf{y})/p(\mathbf{y}_{F_k} \mid \mathbf{y}_{-F_k})$.

Its four full conditionals are

$$(1)\quad \Pr(\kappa = k \mid \theta, \mathbf{W}) \;\propto\; \prod_{i \notin F_k} f(y_i \mid \mathbf{w}_i) \;\propto\; \exp\{-L_k(\mathbf{W})\}, \tag{34}$$

$$\text{with } L_k(\mathbf{W}) = \sum_{i \in F_k} \sum_j \log f_j(y_{ij} \mid w_{ij}), \tag{35}$$

$$(2)\quad \phi \mid \boldsymbol{\beta}, \boldsymbol{\Sigma}, \mathbf{W}, \kappa \;\propto\; p(\phi)\, p_V(\mathbf{W} \mid \boldsymbol{\beta}, \boldsymbol{\Sigma}, \phi), \tag{36}$$

$$(3)\quad \boldsymbol{\Sigma}, \boldsymbol{\beta} \mid \phi, \mathbf{W}, \kappa \;\propto\; p(\boldsymbol{\beta} \mid \boldsymbol{\Sigma})\, p(\boldsymbol{\Sigma})\, p_V(\mathbf{W} \mid \boldsymbol{\beta}, \boldsymbol{\Sigma}, \phi), \tag{37}$$

$$(4)\quad \mathbf{W} \mid \theta, \kappa, \mathbf{y} \;\propto\; p_V(\mathbf{W} \mid \theta) \prod_{i \notin F_\kappa} f(y_i \mid \mathbf{w}_i). \tag{38}$$

Steps (2) and (3) do not involve $\mathbf{y}$ or κ at all: the data and the held-out fold only enter through $\mathbf{W}$ in step (4).

Step in code	Required conditional	Verdict
1. Gumbel-max for κ	(1)	**Differs.** Uses $e^{-\ell_k}$ with the noisy neighbour-based score of Section 9, not $e^{-L_k(\mathbf{W})}$. See Section 10.5.
2a. `update_phi`	(2)	Correct (Metropolis–Hastings with the truncation correction).
2b. `update_Sigma`, `update_beta`	(3)	Correct (conjugate draws; $\boldsymbol{\Sigma}$ first with $\boldsymbol{\beta}$ integrated out, then $\boldsymbol{\beta}$).
2c. `update_W`	(4)	Correct (ESS with training sites only).
3. fold scores	(feeds step 1)	Noisy, independent-site (Section 9).

10.3 The function, line by line

Block A: arguments.

```
List gibbs_kfold_joint_cpp(
    const NumericMatrix& Y, const NumericMatrix& X, const NumericMatrix& locs,
    const IntegerMatrix& NNarray, const IntegerVector& fold, int K,
    const List& test_nn_list,
    const CharacterVector& family, const NumericVector& family_par,
    double nu, double phi_init, const NumericMatrix& beta_init,
    const NumericMatrix& Sigma_init, const NumericMatrix& W_init,
    const NumericMatrix& M_prior, const NumericMatrix& V_prior,
    const NumericMatrix& S_prior, double df_prior,
    double b_phi, double tuning_phi, int niters)
```

Data and structure: $\mathbf{Y}, \mathbf{X}$, coordinates, the Vecchia neighbour matrix `NNarray`, fold labels $1, \ldots, K$ and the held-out neighbour lists. Model: families and their parameters, the fixed ν. Starting values: $\phi^{(0)}, \boldsymbol{\beta}^{(0)}, \boldsymbol{\Sigma}^{(0)}, \mathbf{W}^{(0)}$. Priors: $\mathbf{M}_0, \mathbf{V}_0, \mathbf{S}_0, \delta_0$ and the upper bound b_ϕ. Tuning: the proposal variance for ϕ, and the number of iterations.

Block B: sizes and checks.

```
const int N = Y.nrow(), q = Y.ncol(), p = X.ncol();
if (X.nrow() != N || locs.nrow() != N || NNarray.nrow() != N || fold.size() != N ||
    W_init.nrow() != N || W_init.ncol() != q) stop("Dimension␣mismatch␣between␣inputs.");
if (family.size() != q) stop("length(family)␣must␣equal␣ncol(Y).");
```

Reads N, q, p and stops with an error if the inputs do not line up. No statistics happen here.

Block C: translate the family names.

```
std::vector<int> fam(q);  std::vector<double> par(q, 1.0);
for (int j = 0; j < q; ++j) {
  std::string f = as<std::string>(family[j]);
  if (f == "Gaussian") fam[j] = 0;   else if (f == "Poisson") fam[j] = 1;  ...
  if (family_par.size() == q && !NumericVector::is_na(family_par[j])) par[j] = family_par[j];
}
```

Each family name becomes an integer code (0 Gaussian, 1 Poisson, 2 Binomial, 3 Gamma, 4 Negative-binomial) used inside `loglik1`. The extra parameter `par` defaults to 1 unless a non-`NA` value is given. In `run_M1.R`, `family_par` is all `NA`, so the Gaussian variance is 1.

Block D: recode folds to 0-based.

```
std::vector<int> fold0(N);
for (int i = 0; i < N; ++i) { fold0[i] = fold[i] - 1; if (fold0[i] < 0 || fold0[i] >= K) stop(...); }
```

Converts F_k labels $1, \ldots, K$ to $0, \ldots, K-1$ and checks the range.

Block E: copy to Armadillo matrices; prior matrices.

```
const arma::mat Ya = as<arma::mat>(Y), Xa = as<arma::mat>(X);
arma::mat W = as<arma::mat>(W_init), beta = ..., Sigma = ...;
arma::mat cholSigma = arma::chol(Sigma, "upper");
const arma::mat Mprior = ..., Sprior = ..., Vprior = ...;
const arma::mat precP  = arma::inv_sympd(Vprior);
const arma::mat cholP  = arma::chol(precP, "upper");
```

Sets the current state $(\mathbf{W}, \boldsymbol{\beta}, \boldsymbol{\Sigma})$ and its Cholesky factor, and the prior quantities $\mathbf{M}_0, \mathbf{S}_0$, the prior precision $\mathbf{V}_0^{-1}$ (`precP`) and its Cholesky factor (`cholP`), used in $\widetilde{\mathbf{S}}$ (Section 6.2).

Block F: fixed standard deviations, Vecchia structure and starting coefficients.

```
std::vector<double> fixed_sd(q, 0.0);
for (int j = 0; j < q; ++j) fixed_sd[j] = std::sqrt(Sigma(j, j));
Vecchia V; build_structure(NNarray, V);
Coeff C, Ccand;
double phi = phi_init;
vecchia_coeff_fill(locs, V, phi, nu, C);
```

- `fixed_sd` stores $\sqrt{\Sigma_{jj}^{(0)}}$; it is only used for Binomial responses (not active here).
- `build_structure` converts `NNarray` to 0-based neighbour lists and records where each site itself sits in its row (it stops if the site is missing from its own row).
- `C` holds the Vecchia coefficients $(\mathbf{b}_i, r_i)$ at the current ϕ; `Ccand` is the space for the candidate ϕ^*. `vecchia_coeff_fill` computes `C` at $\phi^{(0)}$.

Block G: output storage and the initial fold scores.

```
const int ntheta = 1 + p * q + q * (q + 1) / 2;
NumericMatrix fold_ll_out(niters, K), theta_out(niters, ntheta);
IntegerVector k_out(niters);
std::vector<double> fold_ll(K);
compute_all_fold_predictive_loglik(Ya, Xa, W, beta, cholSigma, locs, test_nn_list,
                                   phi, nu, fam, par, fold_ll);
std::vector<char> train(N, 1);
```

Allocates the output ($1 + pq + q(q+1)/2 = 10$ columns for θ here), then computes $\ell_1, \ldots, \ell_K$ at the starting state; these are what the *first* draw of κ uses. `train` will mark which sites are used in the current iteration.

Block H: step 1, draw κ.

```
for (int iter = 0; iter < niters; ++iter) {
  double best = R_NegInf;
  for (int l = 0; l < K; ++l) {
    const double v = -fold_ll[l] - std::log(-std::log(unif_rand()));
    if (v > best) { best = v; k = l; }
  }
  for (int i = 0; i < N; ++i) train[i] = (fold0[i] != k);
```

Gumbel-max (Section 5.2): $\kappa = \arg\max_l\{-\ell_l + G_l\}$, $G_l = -\log(-\log U_l)$, giving $\Pr(\kappa = l) \propto e^{-\ell_l}$. Then every site outside fold κ is marked as training.

Block I: step 2, update ϕ, then $(\boldsymbol{\Sigma}, \boldsymbol{\beta})$, then W.

```
M = Xa * beta;
bool acc = false;
update_phi(phi, M, W, cholSigma, locs, V, C, Ccand, nu, b_phi, tuning_phi, acc);
acc_main += acc;

vecchia_products(W, Xa, V, C, UW, UX);
const arma::mat prec_tilde = UX.t() * UX + precP;
const arma::mat V_tilde = arma::inv_sympd(prec_tilde);
const arma::mat Rtilde  = arma::chol(V_tilde, "upper");
const arma::mat Mpart   = UX.t() * UW + precP * Mprior;

update_Sigma(UW, Mprior, cholP, Mpart, Rtilde, Sprior, df_prior, N, fam, fixed_sd, Sigma, cholSigma);
beta = update_beta(Mpart, cholSigma, Rtilde);

M = Xa * beta;
update_W(W, M, Sigma, V, C, Ya, fam, par, train);
```

- $\mathbf{M} = \mathbf{X}\boldsymbol{\beta}$ with the current $\boldsymbol{\beta}$; `update_phi` is the MH step (6.1). If accepted, `C` is replaced by the candidate coefficients, so later lines use the new ϕ.
- `vecchia_products` makes $\mathbf{UW}, \mathbf{UX}$ with the *new* coefficients. Then $\widetilde{\mathbf{V}}^{-1} = \mathbf{UX}^\top\mathbf{UX} + \mathbf{V}_0^{-1}$ (`prec_tilde`), `V_tilde` is its inverse, `Rtilde` its upper Cholesky factor, and `Mpart`$= \mathbf{UX}^\top\mathbf{UW} + \mathbf{V}_0^{-1}\mathbf{M}_0 = \widetilde{\mathbf{V}}^{-1}\widetilde{\mathbf{M}}$.
- $\boldsymbol{\Sigma}$ is drawn first (with $\boldsymbol{\beta}$ integrated out), then $\boldsymbol{\beta} \mid \boldsymbol{\Sigma}$ (6.2). Because $\boldsymbol{\beta}$ changed, $\mathbf{M}$ is recomputed.
- `update_W` runs ESS for each column with the new $\mathbf{M}, \boldsymbol{\Sigma}, \phi$ and with the likelihood of training sites only (6.3).

Block J: step 3, scores and saving.

```
compute_all_fold_predictive_loglik(Ya, Xa, W, beta, cholSigma, locs, test_nn_list,
                                   phi, nu, fam, par, fold_ll);
for (int l = 0; l < K; ++l) fold_ll_out(iter, l) = fold_ll[l];
k_out[iter] = k + 1;
int c = 0;
theta_out(iter, c++) = phi;
for (int j = 0; j < q; ++j) for (int i = 0; i < p; ++i) theta_out(iter, c++) = beta(i, j);
for (int j = 0; j < q; ++j) for (int i = 0; i <= j; ++i) theta_out(iter, c++) = Sigma(i, j);
```

Recomputes $\ell_1, \ldots, \ell_K$ at the new state (used by the next iteration's step 1 and saved). Row `iter` stores $\kappa^{(s)}$ (the fold that was left out when updating), $\theta^{(s)} = (\phi, \operatorname{vec}\boldsymbol{\beta}, \operatorname{upper}(\boldsymbol{\Sigma}))$ and $\ell(\theta^{(s)})$. This pairing is the right one: $\theta^{(s)}$ was drawn *given* $\kappa^{(s)}$, and ℓ is evaluated at that same $\theta^{(s)}$.

Block K: housekeeping and return.

```
if ((iter & 15) == 0) Rcpp::checkUserInterrupt();
if ((iter + 1) % step_10pct == 0 || (iter + 1) == niters) { /* print progress */ }
...
double rate_main = niters > 0 ? (double)acc_main / niters : NA_REAL;
return List::create(_["samples"] = theta_out, _["theta"] = theta_out, _["fold_loglik"] = fold_ll_out,
  _["acc"] = rate_main, ..., _["folds"] = k_out, _["state"] = List::create(W, beta, Sigma, phi));
```

Lets you interrupt in R, prints progress every 10%, and returns the draws, scores, folds, acceptance rate of ϕ (accepted/`niters`) and the final state. There is no burn-in or thinning inside the function.

10.4 Checking each conditional against the target

(2) ϕ — correct. The target is $p(\phi)\, p_V(\mathbf{W} \mid \boldsymbol{\beta}, \boldsymbol{\Sigma}, \phi)$ over all N sites; the code evaluates exactly that with `vecchia_ll` and the uniform prior. The proposal is a truncated normal, and the correction $\log Z(\phi) - \log Z(\phi^*)$ makes the acceptance ratio exact.

(3) $\boldsymbol{\Sigma}, \boldsymbol{\beta}$ — correct. With $\mathbf{UW} = \mathbf{UX}\boldsymbol{\beta} + \mathbf{E}$ the Vecchia likelihood is a matrix-normal regression with N independent rows, so the matrix-normal–inverse-Wishart update with $\delta_0 + N$ degrees of freedom is exact. I also checked that the Bartlett inverse-Wishart draw (`riwish_cpp`) and the matrix-normal draw $\boldsymbol{\beta} = \widetilde{\mathbf{M}} + \mathbf{R}_{\tilde{V}}^{\top} \mathbf{Z} \mathbf{R}_{\Sigma}$ have the right covariances.

(4) $\mathbf{W}$ — correct. Given the other columns, column j has prior $\mathcal{N}(\boldsymbol{\mu}_j, \sigma^2_{j|-j}\, (\mathbf{U}^{\top}\mathbf{U})^{-1})$ (exact conditional of the Vecchia matrix-normal), the prior draw `vecchia_prior_draw` has precisely that covariance, and the likelihood is only over training sites. ESS is therefore valid. A useful side effect: the surface at held-out sites receives no likelihood, so under $\kappa = k$ the block $\mathbf{W}_{F_k}$ is moved jointly and with its full spatial correlation, conditional on everything else.

(1) κ — this is the step that does not match. See next subsection.

10.5 The κ step: where the code departs from q_{mix}

Equation (34) says the exact conditional uses *the current* $\mathbf{W}$ *at the held-out sites*. The code instead uses ℓ_k from Section 9, which (a) ignores $\mathbf{W}_{F_k}$ and draws a fresh $\tilde{\mathbf{w}}_i$ for each held-out site from its training neighbours, (b) uses one draw per site, independently across sites, and (c) evaluates folds $l \neq \kappa$ at a state that has already used their data. So the code samples from an approximation of q_{mix}, not from q_{mix} itself:

- the exponential of a noisy score is not an unbiased stand-in for $1/p(\mathbf{y}_{F_k} \mid \mathbf{y}_{-F_k}, \theta)$ (Jensen);
- because ℓ_l for $l \neq \kappa$ is optimistic (higher) while ℓ_κ is honest, and $\Pr(\kappa = l) \propto e^{-\ell_l}$, the draw is pushed to *stay* on the current fold, so κ may mix slowly and be biased;
- the stored ℓ_k are then used by the estimator, so any mismatch carries into the ELJPD (I cannot see `eljpd.utils.R`).

Exact (noise-free) alternative, within the code's own target. Replace the score by $L_k(\mathbf{W})$ from (34), which needs no extra draws and no Matérn/Cholesky work:

```
std::fill(fold_ll.begin(), fold_ll.end(), 0.0);
for (int i = 0; i < N; ++i)
  for (int j = 0; j < q; ++j)
    fold_ll[fold0[i]] += loglik1(fam[j], par[j], Ya(i, j), W(i, j));
```

(computed at the start, and again after `update_W`). With this choice all four steps are exact Gibbs conditionals of the same $q_{\text{mix}}(\theta, \mathbf{W}, \kappa)$. It also gives a clean identity for the quantity of interest,

$$p(\mathbf{y}_{F_k} \mid \mathbf{y}_{-F_k}) = \mathbb{E}_{p(\theta, \mathbf{W} \mid \mathbf{y}_{-F_k})}\Big[\prod_{i \in F_k} f(y_i \mid \mathbf{w}_i)\Big] = \mathbb{E}_{q_{\text{mix}}(\cdot \mid \kappa = k)}\big[e^{L_k(\mathbf{W})}\big], \tag{39}$$

which the estimator would have to use consistently.

Caveat for the alternative. Under $\kappa = k$, $\mathbf{W}_{F_k}$ carries no data and so fits $\mathbf{y}_{F_k}$ much worse than the other folds, i.e. L_k is much smaller than L_l. Since $\Pr(\kappa = l) \propto e^{-L_l}$, the chain will probably stay on fold k for long stretches (very sticky κ). I expect this from the formula but have not run it. This may be exactly why the code uses a neighbour-based score instead.

10.6 Other things worth checking

- **Neighbour ordering.** `vecchia_prior_draw` builds ξ_i from ξ_l in index order, so every neighbour index in `NNarray` must be *smaller* than i (earlier in the max–min ordering). Please confirm `neighbor_matrix` guarantees this.
- **Fold imbalance.** The marginal of κ is $\propto 1/p(\mathbf{y}_{F_k} \mid \mathbf{y}_{-F_k})$, so hard-to-predict folds are visited more. Look at `table(fit_main$folds)`; the script prints only the number of distinct folds.
- **Silent failure in the κ draw.** If every v is `NaN`, the test `v > best` never succeeds and κ silently keeps its previous value. A non-finite fold score would cause this.
- **Output pairing.** Row s stores $(\kappa^{(s)}, \theta^{(s)}, \ell(\theta^{(s)}))$; I checked that this is the correct pairing for the scan order used.

11 Points worth being ready to explain

- **Fixed quantities.** $\nu = 0.5$ is fixed (exponential covariance); only ϕ is learned. The Gaussian response has observation variance fixed at 1 (`family_par` is `NA`, so it defaults to 1), so the Gaussian variability is carried by $\mathbf{\Sigma}$ and the fixed noise.
- **Why Gumbel-max.** It samples κ with probabilities $\propto e^{-\ell_k}$ without numerical overflow.
- **Noisy, independent-site scores.** Each ℓ_k uses one random draw per held-out site, drawn independently across sites. Held-out sites are not correlated with each other in the score, and the fresh noise can hide MCMC autocorrelation (ESS close to S). See Section 9.
- **Held-out sites inside W.** $\mathbf{W}$ is kept at *all* N sites; held-out sites simply contribute no likelihood in that iteration. The scoring function does not read those values.
- **Possible leakage for $l \neq \kappa$.** Scores of folds other than the one currently left out use a $\mathbf{W}$ that has already seen those folds' data (my reading; to be confirmed). See Section 10.5.
- **ESS safeguard.** The slice-sampling loop is capped at 1000 shrinks; if it ever hit the cap, that column of $\mathbf{W}$ would be left unchanged for that iteration (this should essentially never happen).
- **Not checked here.** I could not run the sampler: `vecchia.R`, `aux_functions_joint.R`, `eljpd.utils.R` and `data_full.RData` were not provided. Everything above is derived by reading the code.